

Exhaustive Technical Report: GenAI Predictive Maintenance in an Edge-Fog-Cloud Architecture

1. Introduction and Objectives

In modern Industrial Internet of Things (IIoT) deployments, physical machinery generates an astronomical volume of telemetry (e.g., temperature, continuous positional tracking, vibration frequencies, current load). Traditionally, this data was piped entirely into a centralized Cloud warehouse for batch analysis. However, this centralized approach induces crippling latency overheads, exorbitant bandwidth costs, and single points of absolute failure.

This project implements a state-of-the-art **Edge-Fog-Cloud** hierarchy. By physically distributing compute power, the architecture is able to detect mechanical anomalies instantaneously at the machine level (Edge), aggregate and enrich regional alerts on the factory floor (Fog), and perform mathematically intensive, multidimensional Machine Learning and Generative AI diagnostics globally (Cloud). This report details the precise inner workings of every script, protocol, and algorithm composing this simulation.

2. The Communication Protocol: MQTT

Relevant Files: `docker-compose.yml`, `mosquitto.conf`

Before analyzing the nodes, one must understand the communicative fabric connecting them. The system explicitly rejects standard HTTP REST patterns in favor of **Message Queuing Telemetry Transport (MQTT)**. - **Why MQTT?** It operates on a Publish/Subscribe (Pub/Sub) model. Standard REST requires heavy TCP/HTTP headers per transmission. MQTT features a 2-byte header, making it the industry standard for constrained industrial networks. - **The Broker:** We utilize an `eclipse-mosquitto` container (port 1883). The broker acts as a central post office; it does not process data, it strictly routes topics. If an edge agent dies, the broker persists, allowing the system to naturally heal and decouple data generation from data consumption.

3. Data Generation: The Machine Simulators

Relevant File: `scripts/mqtt_simulator.py`

In the absence of physical turbofan engines or factory hydraulic pumps, this script mathematically synthesizes realistic machine behaviors. - **Target Fleet:** Loops across 5 devices (`device-01` to `device-05`). - **Data Profiles:** The simulator establishes baseline metrics for each device utilizing Gaussian (Normal) distributions. - *Temperature:* Baselines around 45.0°C. - *Vibration (RMS):*

Baselines around 1.0. - *Current*: Baselines around 2.0 Amps. - **Continuous Injection**: A perpetual `while True`: loop fires a complete metric snapshot every 2 seconds. - **Algorithmic Anomaly Injection**: Machine learning requires failure data. We utilize Python's `random.random()` to mathematically force failure. There is a 2% chance per tick to artificially inflate the temperature reading by up to 20°C and mark `injected_anomaly=1`. This creates the statistical outliers our systems are built to catch. - **Egress**: The dictionary is serialized into a JSON string and published directly to `factory/<device_id>/telemetry`.

4. The Data Ingestion Engine

Relevant Files: `scripts/mqtt_ingest.py`, `src/db.py`

The Cloud dashboard must query historical data. To bridge the gap between real-time UDP-style telemetry streams and historical analytical queries, we employ a dedicated headless ingestor. - **The Callback Cycle (`mqtt_ingest.py`)**: The script subscribes broadly to the wildcard topic `factory/+/telemetry`. The `on_message` callback fires every time the broker receives a packet. It decodes the JSON payload, coerces it into a Pandas DataFrame, casts the standard ISO8601 timestamp into a datetime object, and passes it to the database module. - **The Storage Schema (`db.py`)**: Leveraging Python's native SQLite3, it initializes `telemetry.db`. To guarantee that the analytical frontend does not bottleneck when querying hundreds of thousands of rows, `db.py` rigidly establishes an SQL index spanning the composite key (`device_id`, `timestamp`). - **Graceful Insertion**: `df_db.to_sql("telemetry", conn, if_exists="append", index=False)` allows rapid, continuous appends.

5. The Edge Agent: Latency-Free Heuristics

Relevant File: `src/edge_agent.py`

This script represents code running directly on the microcontroller of the industrial machine. - **Stream Interception**: It actively intercepts the telemetry stream for its device. - **O(1) Evaluation Loop**: For every packet, it evaluates the keys against an internal `THRESHOLDS` dictionary (e.g., Temperature > 60.0). This operates in O(1) constant time, requiring almost zero CPU cycles. - **Data Decimation**: If no threshold is breached, the data is entirely ignored. This represents the primary advantage of Edge Computing: we do not waste bandwidth on 'normal' operational metrics. - **Alert Dispatch**: If an anomaly is located, a compressed subset of data (identifying only the breached metric) is re-routed to a highly critical communication channel: `factory/<device_id>/alerts`. - **Command Control**: The agent subscribes dually to `factory/+cmd/firmware`. This allows overriding the hardcoded

Python thresholds without interrupting the running process, directly answering the need for remote fleet management.

6. The Fog Gateway: Floor-Level Aggregation

Relevant File: `fog_app.py`

Deployed physically close to the edge nodes (e.g., a server in a factory warehouse), the Fog application manages immediate local networks. - **Asynchronous Architecture:** Streamlit typically struggles with background loops due to its reactive render cycle. `fog_app.py` circumvents this utilizing the `@st.cache_resource` decorator to spawn a permanent background daemon string. This daemon maintains a constant MQTT connection independent of the UI redraws. - **Data Enrichment:** When raw alerts drop in from the edge (e.g., `device-04`), the Fog node performs simple lookup logic, appending geospatial tracking strings (e.g., `Factory-Floor-A (Rack 04)`) to the payload. - **Local Persistence:** It leverages JSON-Lines (`.jsonl`) to append these alerts safely to the local disk, averting multi-threading race conditions while making the data available to local administrators visually via the `st.dataframe` module.

7. Cloud Analytics: Multi-Dimensional ML Isolation

Relevant Files: `app.py`, `src/anomaly.py`

The primary interface functions as the central nervous system. - **The ML Engine (IsolationForest):** Simple thresholds (used on the Edge) fail to capture multivariable drift (e.g., Temperature rising slightly while Current drops slightly). `src/anomaly.py` ingests the historical telemetry from SQLite and executes a Scikit-Learn Isolation Forest. - *Feature Scaling:* We utilize `RobustScaler()` to nullify severe outliers during the normalization phase. - *Tree Building:* The forest generates 250 decision trees (`n_estimators=250`), attempting to arbitrarily split the dataset. Anomalous points conceptually require fewer splits to isolate than clustered normal behavior. - *Scoring:* We extract the Scikit-Learn `decision_function` array (where normal behavior yields positive values) and invert it, ensuring **higher = more anomalous** for visual intuition. - *Explainability Proxies:* Isolation Forests are notorious black boxes. To explain *why* the model failed a row, we dynamically calculate the **Median Absolute Deviation (MAD)** per feature. We distribute the holistic anomaly score proportionately based on these robust z-scores to determine which specific sensor contributed most heavily to the failure.

8. Generative AI Diagnostics & RAG

Relevant Files: `src/rag.py`, `src/llm.py`

Identifying a failure does not inherently fix a machine. To aid technicians, the project utilizes Retrieval-Augmented Generation (RAG). - **Document Chunking** (`pypdf`, `chunk_text`): Users upload pure PDF machine maintenance manuals. `rag.py` parses the documents, stripping excess whitespace, and breaks the text into 900-character segments utilizing a 140-character sliding overlap (to prevent chopping critical sentences in half). - **Vector Embedding**: Using HuggingFace’s `sentence-transformers/all-MiniLM-L6-v2` dense embedding model, these text chunks are transformed into 384-dimensional mathematical vectors. - **FAISS Storage**: These vectors are permanently indexed in Meta’s **Facebook AI Similarity Search (FAISS)** database, utilizing `faiss.IndexFlatIP` (Inner Product / Cosine Similarity). This allows the application to query thousands of textbook pages in milliseconds. - **LLM Synthesis**: When an anomaly is detected, the top 5 most mathematically relevant snippets of physical manuals are yanked from FAISS. They are injected into a hardcoded System Prompt referencing the live data anomalies. - **Execution** (`llm.py`): This prompt is shipped via HTTP requests to either a localized (free & private) **Ollama** engine (running `llama3.2:3b`) or to external routed endpoints (OpenRouter). The Large Language Model processes the telemetry against the manual chunks, outputting a precise, numbered, citation-backed maintenance protocol avoiding hallucinatory guesses.

9. Security & Over-The-Air (OTA) Updates

To truly function as an IoT simulation, the cloud must be able to push state down to the hardware. - **The Update UI**: The `app.py` “OTA Updates” tab captures raw integer limits from human operators. - **Payload Construction & Security**: The Python code constructs an update packet and signs the integer with `hashlib.sha256(str(new_temp_threshold).encode()).hexdigest()`. While simplified in this simulation, this architecture perfectly mirrors signed firmware verification schemas. - **Execution**: The script executes an immediate `pub_client.publish()` blast against the target `cmd/firmware` topic, completing the cyber-physical loop as the specific `edge_agent.py` processes it.

10. Conclusion

By fracturing compute operations across Edge agents, Fog routing gateways, and specialized Cloud containers, this simulation successfully mitigates the severe bandwidth constraints intrinsic to heavy industry, minimizes anomaly detection latency via Edge heuristics, enables localized routing tracking via the Fog, and surfaces the true power of Generative AI diagnostics globally at the Cloud level.